

Antarctic Digital Twin Reference Manual

Technical Feature Specification & System Architecture — Stations Bharati & Maitri

1. Dynamic Fuel & Physics Engine

- Feature Name & File Context:** Dynamic Fuel & Physics Engine
(`src/engine/fuelPhysicsEngine.js`)
- What It Does:** Calculates real-time polar diesel consumption rates and predicts fuel depletion risks based on ambient Antarctic temperatures and active generator operational loads.
- Why We Need It:** Extreme cold (-40°C to -60°C) dramatically increases fuel viscosity and heating overhead, making standard burn rate predictions inaccurate and putting station survival at risk during harsh winter months.
- How It Works & Math Logic:** The engine updates every 3000 milliseconds (3 seconds). Base fuel consumption is set to 45.0 liters per hour per active generator. When ambient temperature drops below -20°C, the cold multiplier applies: $\text{coldMultiplier} = 1 + (|\text{ambientTemperature}| - 20) \times 0.018$. If temperature drops below -45°C, fuel viscosity risk flag is set to true. Total station burn rate is calculated as: $\text{actualBurnRate} = \text{baseBurnRate} \times \text{activeGenerators} \times \text{coldMultiplier} \times \text{scenarioPenaltyMultiplier}$. The remaining survival window in days is determined by: $\text{survivalWindowDays} = \text{totalRemainingFuelLiters} / (\text{actualBurnRate} \times 24)$. If survivalWindowDays drops below 15.0 days, a CRITICAL_FUEL_WARN alert is triggered.
- Tech Stack:** React custom hooks (`useFuelEngine`), JavaScript ES6 math utilities, Tailwind CSS for status badges, Lucide React for warning icons (`Flame` , `AlertTriangle` , `Gauge`).
- Data Source (Prototype Mock Data):** Hardcoded baseline initial storage of 120,000 liters across 4 tanks, initialized via React `useState` , with local state timers mimicking consumption reduction every 3 seconds.
- Data Source (Real-World Implementation):** High-accuracy radar liquid level sensors (e.g., Endress+Hauser Micropilot) installed in storage tanks, connected via Modbus RTU/TCP to an OPC UA industrial server, streamed into InfluxDB time-series database.

2. Interactive 2D Station Layout Map

- Feature Name & File Context:** Interactive 2D Station Layout Map (`src/components/StationMap2D.jsx`)
- What It Does:** Renders a real-time vector floor plan of Bharati and Maitri stations with color-coded operational zones, module temperatures, and generator status indicators.
- Why We Need It:** Station commanders need immediate spatial awareness to pinpoint thermal leaks, power grid failures, or isolated module shutdowns across sprawling habitats.

- **How It Works & Math Logic:** Renders SVG blueprint elements mapped to grid coordinates (X, Y, Width, Height). Each zone evaluates its operational state dynamically: if zone temperature falls below 15.0°C or power load exceeds 90.0%, status switches to WARNING (amber color #f59e0b). If temperature falls below 5.0°C or power drops to 0kW while assigned active, status switches to CRITICAL (red color #ef4444). Otherwise, status is NORMAL (green color #10b981). Interactive clicks trigger state callbacks selecting the zone ID into global application context for detailed diagnostic overlay rendering. Updates every 1000 milliseconds.
- **Tech Stack:** React, SVG (Scalable Vector Graphics), Tailwind CSS styling, Lucide React icons (`Zap` , `Thermometer` , `ShieldAlert` , `Activity`).
- **Data Source (Prototype Mock Data):** Static JSON configuration array (`src/data/stationZones.json`) mapping module IDs (e.g., Main Habitat, Generator Room, Lab Pod A) to baseline X/Y SVG path coordinates and default temperature properties.
- **Data Source (Real-World Implementation):** Building Management System (BMS) telemetry over BACnet/IP protocol or MQTT broker collecting data from distributed PT100 temperature sensors and energy meters throughout station modules.

3. Emergency 'What-If' Scenario Simulator

- **Feature Name & File Context:** Emergency 'What-If' Scenario Simulator (`src/components/ScenarioSimulator.jsx`)
- **What It Does:** Allows station operators to trigger extreme emergency events (e.g., Blizzard Level 5, Primary Generator Failure, Fuel Line Freeze) and instantly simulate systemic impact on power and survival timelines.
- **Why We Need It:** Enables proactive crisis planning and testing of emergency contingency protocols without risking actual personnel safety or equipment integrity in harsh polar environments.
- **How It Works & Math Logic:** The user selects a scenario preset or adjusts risk sliders. Scenario multipliers apply as follows: Blizzard Level 5 forces ambient temperature to -55.0°C and applies **scenarioPenaltyMultiplier = 1.45** (representing maximum HVAC heating load). Primary Generator Failure sets active units count from 3 down to 1, applying **generatorFailurePenalty = 2.0** multiplier to remaining units to compensate for load sharing up to maximum cap of 110% capacity. Survival window recalculation logic: **simulatedBurnRate = baseBurnRate × activeGenerators × coldMultiplier × scenarioPenaltyMultiplier**. Reduced target days = **totalRemainingFuelLiters / (simulatedBurnRate × 24)**. If simulatedBurnRate exceeds max output capability, thermal loss rate logic activates: **indoorTempDropRate = (|ambientTemperature| - indoorTemp) × 0.05 °C/hour**. State changes apply immediately upon user interaction.
- **Tech Stack:** React `useContext` and `useReducer` state management, Tailwind CSS controls and toggle switches, Lucide React icons (`AlertOctagon` , `Wind` , `PowerOff` , `RefreshCw`).
- **Data Source (Prototype Mock Data):** Hardcoded preset scenarios array (`src/data/scenarios.js`) defining key-value override pairs for ambient temperature, active generator count, wind speed, and fuel line status.

- **Data Source (Real-World Implementation):** Historical Antarctic climate dataset models combined with real-time digital twin physics solvers running on edge servers (e.g., NVIDIA Jetson or local station server) utilizing historical weather logs from IMD (India Meteorological Department).

4. Central Command Dashboard & Live Analytics

- **Feature Name & File Context:** Central Command Dashboard & Live Analytics (`src/components/CommandDashboard.jsx`)
- **What It Does:** Displays aggregated station telemetry, operational KPIs, real-time power generation vs. load charts, and environmental trend lines for both Bharati and Maitri stations.
- **Why We Need It:** Consolidates fragmented telemetry into a single intuitive operational dashboard, empowering station commanders to make rapid, data-driven survival decisions.
- **How It Works & Math Logic:** Ingests continuous telemetry streams and appends data points every 2000 milliseconds to a rolling 30-entry buffer array. Total power demand is calculated as: $\text{totalPowerKW} = \sum (\text{zonePowerLoadKW})$ across all active zones. System efficiency index is computed as: $\text{efficiencyIndex} = (\text{totalPowerKW} / (\text{activeGenerators} \times \text{maxKWPerGen})) \times 100$. When ambient temperature drops below -35°C , efficiency index is adjusted: $\text{adjustedEfficiency} = \text{efficiencyIndex} \times (1 - (|\text{ambientTemp}| - 35) \times 0.005)$. Alerts trigger if efficiencyIndex exceeds 92.0% (OVERLOAD_WARNING) or drops below 40.0% (INEFFICIENT_RUN_WARNING). Recharts re-renders rolling line and bar graphs dynamically as state updates.
- **Tech Stack:** React, Recharts library (`ResponsiveContainer` , `LineChart` , `BarChart` , `XAxis` , `YAxis` , `Tooltip`), Tailwind CSS grid layout, Lucide React icons (`LayoutDashboard` , `TrendingUp` , `Server` , `Radio`).
- **Data Source (Prototype Mock Data):** Client-side dynamic generator using JavaScript `setInterval` that appends small random Gaussian fluctuations ($\pm 1.5^{\circ}\text{C}$, $\pm 2.5 \text{ kW}$) to baseline parameters stored in React state.
- **Data Source (Real-World Implementation):** Live REST API and WebSocket connections to Open-Meteo weather API targeting station coordinates (Bharati: -69.4089, 76.1914; Maitri: -70.7667, 11.7333) merged with MQTT telemetry from station SCADA systems.